JobMatchHire

NAME:A.KARISHMA **REG NO:**23MIC0057

1. TITLE

JobMatchHire is a resume and job matching platform designed to improve recruitment evaluation through contextual competency analysis, transparent scoring, and candidate-specific developmental feedback.

The platform consists of two independently operating web applications:

Recruiter Portal: Enables recruiters to publish job requirements, view ranked candidate profiles, and inspect competency evidence.

Candidate Portal and ECCE Engine: Enables candidates to upload resumes, evaluate their suitability for selected positions, understand their match scores, and receive private recommendations for improving their profiles.

The platform incorporates the Explainable Contextual Competency Engine (ECCE), which evaluates the actual context in which technical skills are demonstrated rather than relying exclusively on keyword occurrence.

2. MOTIVATION

Recruitment platforms increasingly depend on automated Applicant Tracking Systems (ATS) to process resumes and identify suitable candidates. Although these systems reduce manual screening efforts, conventional approaches still have limitations concerning contextual skill evaluation, transparency, and candidate feedback.

JobMatchHire was developed to address the following problems.

2.1 LACK OF TRANSPARENT CANDIDATE FEEDBACK

Candidates frequently receive application outcomes without sufficient information about why their resumes were considered unsuitable for a particular position. A simple match percentage does not explain which competencies contributed to the score or which requirements were not demonstrated.

JobMatchHire addresses this issue by providing candidates with a detailed evaluation, including matched competencies, missing requirements, scoring explanations, and hypothetical improvement scenarios.

2.2 LIMITATIONS OF KEYWORD-BASED RESUME MATCHING

Traditional keyword-based matching systems primarily identify whether particular technical terms appear in a resume. However, mentioning a technology does not necessarily demonstrate practical experience.

For example, a candidate mentioning Python in a skills section may not have demonstrated the ability to develop an application using Python.

JobMatchHire uses contextual competency evaluation to examine supporting resume passages, action verbs, project descriptions, and measurable outcomes. This helps distinguish demonstrated experience from isolated skill mentions.

2.3 LIMITED PRIVACY IN RECRUITMENT PLATFORMS

Recruitment and candidate development require different types of information. Recruiters need relevant competency evidence to evaluate applicants, whereas candidates may need private feedback about their weaknesses and possible improvements.

Combining these responsibilities within one application can create unnecessary exposure of sensitive candidate information.

JobMatchHire addresses this concern through two independently operating applications and separate databases. Candidate-specific learning recommendations and security tokens remain within the Candidate Portal and are not included in recruiter-facing responses.

2.4 NEED FOR ACTIONABLE SKILL IMPROVEMENT

Most resume evaluation systems focus on the current match score rather than helping candidates understand how their profiles could improve.

JobMatchHire introduces counterfactual simulations that demonstrate how the score could change under hypothetical conditions, such as acquiring a missing skill or clarifying existing experience in the resume.

These simulations help candidates understand potential improvement opportunities without presenting hypothetical results as actual achievements.

3. EXISTING GAP, MOTIVATING EVENT AND UNIQUE SELLING PROPOSITION (USP)

3.1 EXISTING SYSTEM AND RESEARCH GAP

Conventional recruitment platforms commonly rely on keyword filtering, structured eligibility conditions, and automated ranking mechanisms. Although these methods are useful for processing large application volumes, they do not always provide sufficient contextual evidence or actionable explanations.

The following gaps motivated the development of JobMatchHire:

Keyword-Based Evaluation. A conventional keyword matching mechanism may assign relevance to a technical skill simply because the term appears in a resume. It may fail to distinguish between practical implementation experience and a list of technologies.

Black-Box Evaluation. Some automated recruitment systems provide scores or recommendations without explaining the underlying factors. Candidates may not understand which job requirements affected their evaluation.

Absence of Counterfactual Feedback. Most conventional matching workflows do not explain how specific changes in a candidate's demonstrated competencies or resume presentation could affect the calculated score.

Insufficient Privacy Separation. Recruiter evaluation and candidate development have different information requirements. Combining these responsibilities without clear access boundaries can expose private candidate information unnecessarily.

Limited Resume Version Tracking. Candidates may submit updated resumes, but they need a systematic way to understand how their evaluation changes across different submissions.

3.2 UNIQUE SELLING PROPOSITIONS (USP)

JobMatchHire introduces the following features to address these gaps.

Decoupled Two-Application Architecture. The system contains two independent FastAPI applications: Application A: Recruiter Portal, using version_a.db; Application B: Candidate Portal and ECCE Engine, using version_b.db. The applications communicate through REST APIs using HTTP and JSON.

Contextual Competency Extraction. ECCE examines supporting resume sentences to identify action verbs such as developed, engineered, deployed, optimized, and architected; project contexts such as APIs, microservices, backend systems, pipelines, and infrastructure; and quantified outcomes such as performance improvements and request-handling capacity.

Three-Way Evidence Classification. Each assessed competency is assigned one of three classifications: Evidence Found; Evidence Not Found; and Potential Inconsistency. Evidence Not Found means supporting evidence was not identified in the submitted resume; it does not prove that the candidate lacks the skill. Potential Inconsistency means a skill appears as an isolated keyword without sufficient supporting project evidence. This classification requires review and must not be interpreted as proof of dishonesty.

Deterministic Weighted Scoring. The matching engine uses a visible scoring formula: Required Skills 50%, Experience 25%, Preferred Skills 15%, and TF-IDF Cosine Similarity 10%. This

makes the scoring mechanism understandable and reproducible for the same inputs and evaluation configuration.

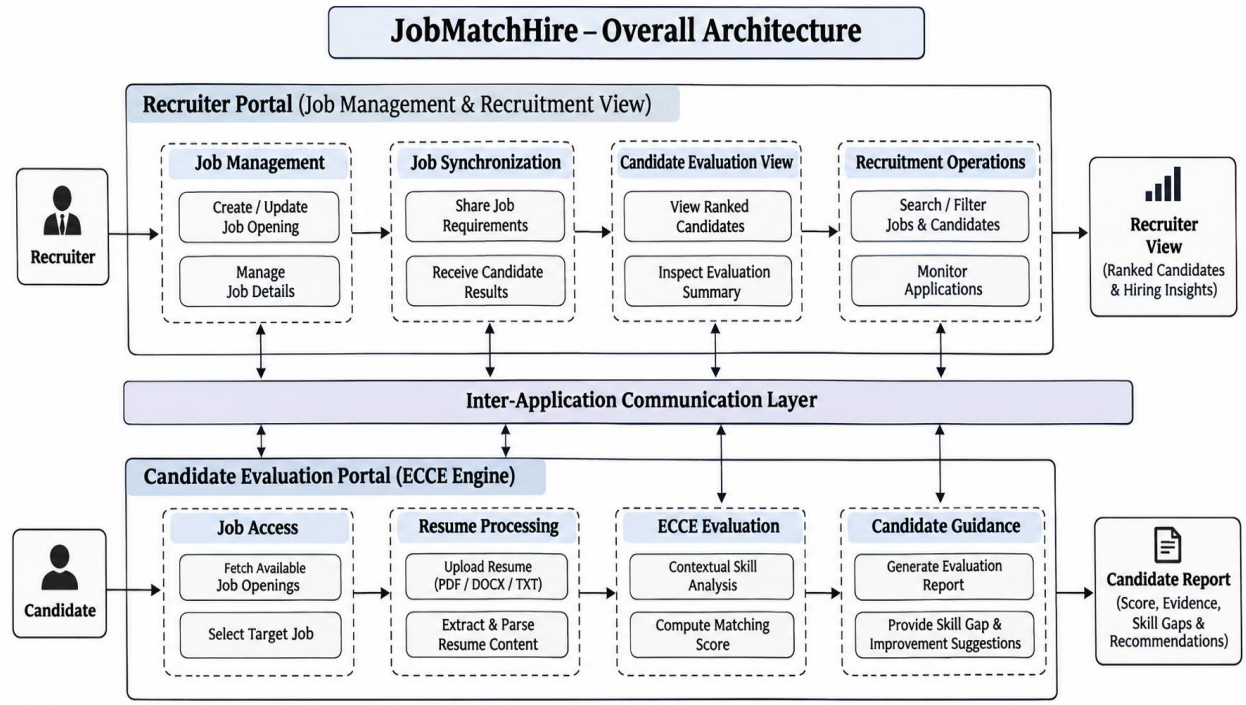
Counterfactual Sensitivity Simulations. Candidates can explore hypothetical scenarios involving missing competencies or clarification of existing experience. The system distinguishes between skill acquisition, meaning gaining practical experience through actual learning and project work, and resume correction, meaning clearly documenting experience that already exists. Every scenario carries the label: Hypothetical Simulation – Not Actual Result.

Private Candidate Evaluation Reports. The Candidate Portal generates an eight-section PDF evaluation report containing scoring details, competency evidence, explanations, hypothetical simulations, and private learning guidance. Access is controlled using a cryptographically random candidate token.

Resume Versioning and Iterative Evaluation. The platform tracks candidate resume versions, such as v1, v2, and subsequent submissions. Candidates can compare their evaluation results as they update their resumes.

Recruiter-Candidate Privacy Firewall. Only recruiter-safe candidate information is transmitted to the Recruiter Portal. Private learning recommendations, candidate security tokens, and developmental guidance are excluded from recruiter-facing payloads. Restricted candidate-history endpoints return HTTP 403 Forbidden.

4. ARCHITECTURE



JobMatchHire follows a distributed two-application architecture. Both applications use Python and FastAPI but maintain separate databases and responsibilities.

4.1 OVERALL SYSTEM ARCHITECTURE

Application A – Recruiter Portal

FastAPI backend.
Job posting and requisition management.
Candidate leaderboard generation.
Recruiter evidence inspection.
SQLite database: version_a.db.

Application B – Candidate Portal and ECCE Engine

FastAPI backend.
Resume parsing and text extraction.
Skill matching and contextual competency evaluation.
Candidate scoring and counterfactual simulations.
ReportLab PDF generation.
SQLite database: version_b.db.
Communication Architecture

Application A sends job requirements to Application B through HTTP/JSON REST API requests. After evaluating a resume, Application B sends recruiter-safe candidate evaluation results back to Application A. Google Gemini is optionally integrated into Application B to assist with contextual interpretation. It does not replace the deterministic scoring mechanism.

5. COMPLETE TECHNOLOGY STACK INFORMATION

5.1 BACKEND AND MICROSERVICE INFRASTRUCTURE

Technology	Version / Specification	Purpose
Python	3.11+	Primary programming language
FastAPI	0.115.0+	Backend development and REST API implementation
Uvicorn	0.30.0+	ASGI server for running FastAPI applications
Pydantic	2.0+	Request validation and structured data handling
HTTPX	0.27.0+	Communication between the two applications
python-dotenv	1.0.0+	Environment variable and API-key management

5.2 DOCUMENT PARSING, NLP AND INFORMATION RETRIEVAL

Technology	Purpose
pdfplumber	Extracting text from PDF resumes
pypdf	Alternative PDF text extraction
python-docx	Extracting paragraphs and tables from DOCX files
Python codecs	Supporting multiple text encodings
Scikit-learn	Text vectorization and similarity calculation
TfidfVectorizer	Converting text into TF-IDF feature vectors
cosine_similarity	Calculating similarity between job descriptions and resume content
Python re module	Skill identification, pattern matching, and experience-date extraction

5.3 EXPLAINABILITY AND PDF GENERATION

ReportLab (4.0.0+)

Used to generate structured candidate evaluation reports in PDF format. The reports contain scoring breakdowns, evidence classifications, supporting passages, and counterfactual simulations.

Python zoneinfo

Used for date and time localization, particularly Indian Standard Time (Asia/Kolkata), in evaluation reports and timestamps.

5.4 DATABASE AND STORAGE

SQLite3

SQLite is used as the relational database engine for both applications.

Database	Stored Information
version_a.db	Job postings, recruiter information, and incoming candidate evaluation summaries
version_b.db	Job requirements, resume text, extracted skills, candidate evaluations, resume versions, and security tokens

Custom database indices support efficient retrieval of candidate rankings using job identifiers and descending match scores.

5.5 FRONTEND TECHNOLOGIES

Technology	Purpose
HTML5	Webpage structure and semantic forms
CSS3	Responsive styling and interface design
Vanilla JavaScript (ES6+)	Frontend interactions and dynamic content
Fetch API	REST API communication
FormData	Resume file uploads
Flexbox and CSS Grid	Responsive page layouts
CSS Custom Properties	Reusable interface design variables

The Recruiter Portal provides job posting interfaces, ranked candidate tables, and evidence inspection modals. The Candidate Portal provides resume upload controls, job selection, score visualizations, and secure report downloads.

5.6 EXTERNAL AI INTEGRATION

Google Gemini 3.8 Flash
Python integration library: google-generativeai, as specified in the project implementation.
Model identifier: gemini-3.8-flash.
Authentication through the GEMINI_API_KEY environment variable.
Used for contextual interpretation of skill assessments.

Gemini assists in evaluating action descriptions, project context, and evidence classifications. The system retains deterministic scoring and a native parsing fallback if the external API is unavailable.

5.7 DEPLOYMENT AND HOSTING

Render Cloud Application Platform
The project uses Render for cloud deployment, with render.yaml for deployment configuration.
Health-check endpoints support service monitoring and connectivity verification between the two applications.
The project also uses environment variables for service URLs and API credentials.

6. DATASETS USED

The JobMatchHire platform does not use any external or pre-existing dataset. It does not depend on a fixed collection of resumes or job descriptions during deployment.

The platform processes resumes uploaded by candidates and job requirements created by recruiters dynamically. A custom competency dictionary, maintained in skills_dict.py, supports skill identification and competency matching.

Therefore, the deployed system operates on user-provided data rather than a predefined dataset.

7. API INTEGRATION

JobMatchHire uses REST APIs to coordinate communication between its two independent applications.

7.1 INTER-SERVICE MICROSERVICE COMMUNICATION

Application A and Application B exchange structured information through HTTP requests and JSON payloads.

Source	Target	Method and Endpoint	Purpose
App A	App B	POST /jobs	Synchronizes newly created job requirements
App B	App A	POST /match-results	Sends candidate evaluation summaries
App A / B	Peer	GET /jobs	Retrieves and synchronizes job information
App A / B	Peer	GET /health	Checks application health and connectivity

Job synchronization: When a recruiter creates a job, Application A stores the requisition in its database and sends the job information to Application B.

Evaluation synchronization: When a candidate submits a resume, Application B evaluates the resume and sends the recruiter-safe result to Application A. The Recruiter Portal then updates the candidate leaderboard.

7.2 RECRUITER PORTAL APIS (APPLICATION A)

Method	Endpoint	Function
GET	/ or /recruiter	Displays the Recruiter Dashboard
GET	/health	Checks application and peer connectivity
POST	/jobs	Creates a job posting
GET	/jobs	Retrieves active job postings
GET	/jobs/{job_id}	Retrieves individual job details
POST	/jobs/{job_id}/sync	Retries job synchronization
DELETE	/jobs/{job_id}	Deletes a job and associated candidate results
POST	/match-results	Receives candidate evaluation results
GET	/jobs/{job_id}/candidates	Retrieves ranked candidates
GET	/jobs/{job_id}/candidates/{candidate_id}/details	Retrieves recruiter-safe evidence details

Candidates are ranked in descending order of their calculated match scores.

7.3 CANDIDATE PORTAL APIS (APPLICATION B)

Method	Endpoint	Function
GET	/ or /portal	Displays the Candidate Portal
GET	/health	Checks application and database health
POST	/analyze-resume	Uploads and evaluates a resume
GET	/candidates/{candidate_id}/report?token={token}	Downloads the authorized PDF report
GET	/candidates/{candidate_id}/status?token={token}	Checks candidate evaluation status
GET	/candidates	Blocks public candidate-history access with HTTP 403

The /analyze-resume endpoint is the primary candidate evaluation endpoint. It accepts a resume file and job identifier, extracts the resume text, performs skill matching, invokes ECCE

contextual analysis, calculates the match score, generates the PDF report, and sends the recruiter-safe evaluation summary to Application A.

7.4 EXTERNAL API INTEGRATION: GOOGLE GEMINI

The project integrates the Google Gemini API to provide additional contextual interpretation.

Parameter	Description
Provider	**Google**
Model	**Gemini 3.8 Flash**
Integration library	**google-generativeai**
Authentication	**GEMINI_API_KEY**
Main purpose	**Contextual interpretation of extracted competency evidence**

Gemini helps assess whether resume passages contain relevant actions and project context. It is not responsible for independently deciding the final numerical score.

If the external service is unavailable, the system uses its native parsing and fallback mechanisms.

8. USERS, INPUTS AND OUTPUTS

8.1 TARGET USERS

A. Corporate Recruiters and Talent Acquisition Teams

Recruiters use the Recruiter Portal to:
Create and manage job postings.
Define mandatory and preferred competencies.
Specify minimum experience and educational requirements.
View ranked candidate leaderboards.
Inspect contextual competency evidence.
Review recruiter-safe evaluation summaries.
Recruiters cannot access private candidate learning recommendations or candidate security tokens.

B. Job Candidates and Applicants

Candidates use the Candidate Portal to:
Select a target job.
Upload their resumes.
View calculated match scores.
Understand matched and missing competencies.
Examine the scoring breakdown.

Explore hypothetical improvement scenarios.
Download private ECCE evaluation reports.
Track resume versions and evaluation changes.

8.2 INPUTS: TYPES AND FORMATS

The platform accepts structured job information, candidate documents, and authentication information.

Input	Format	Source	Description
Job Requisition	JSON	Recruiter Portal	Job title, description, required skills, preferred skills, experience and education
Candidate Resume	PDF, DOCX, TXT	Candidate Portal	Resume document submitted for evaluation
Candidate Overrides	Multipart Form Data	Candidate Portal	Optional candidate name and candidate ID
Job Identifier	String	Candidate Portal	Identifies the selected job
Security Token	URL query parameter	Candidate Portal	Authorizes private report access

Job requisition example:

```
{
  "job_id": "JOB-101",
  "job_title": "Full Stack Developer",
  "job_description": "Develop and maintain web applications",
  "required_skills": ["Python", "FastAPI", "SQL"],
  "preferred_skills": ["React", "Docker"],
  "min_experience": 2.0,
  "education_requirement": "Bachelor's Degree"
}
```

8.3 OUTPUTS: TYPES AND FORMATS

The system generates separate outputs for recruiters and candidates.

A. Candidate Ranked Leaderboard

User: Recruiter.
Format: Interactive HTML table and JSON response.
API: GET /jobs/{job_id}/candidates.
The leaderboard contains:
Candidate rank.
Candidate ID.
Candidate name.

Match score.
Experience status.
Matched skills.
Submission timestamp.
Candidates are sorted according to their calculated match scores.

B. Recruiter Evidence Inspection Modal

User: Recruiter.
Format: Interactive modal and JSON response.
API: GET /jobs/{job_id}/candidates/{candidate_id}/details.
It displays:
Verified competency counts.
Contextual action evidence.
Extracted outcome metrics.
Experience verification status.
Overall evaluation status.
Private learning recommendations are excluded.

C. Candidate Evaluation Dashboard

User: Candidate.
Format: Responsive web dashboard and JSON evaluation response.
API: POST /analyze-resume.
The dashboard includes:
Overall match score displayed through a radial progress meter.
Required Skills score out of 50.
Experience score out of 25.
Preferred Skills score out of 15.
Semantic Similarity score out of 10.
Matched and missing competency indicators.
Counterfactual simulation cards showing hypothetical score changes.
Resume version information, such as v1 or v2.

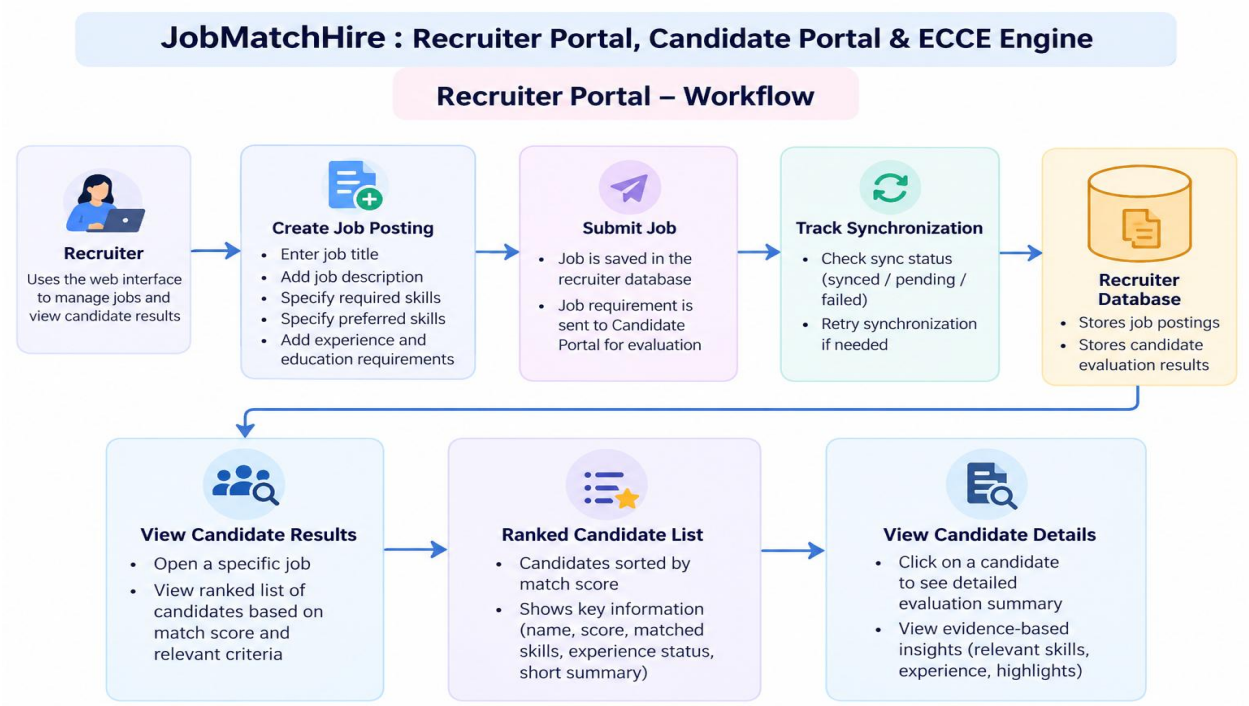
D. ECCE Evaluation Report

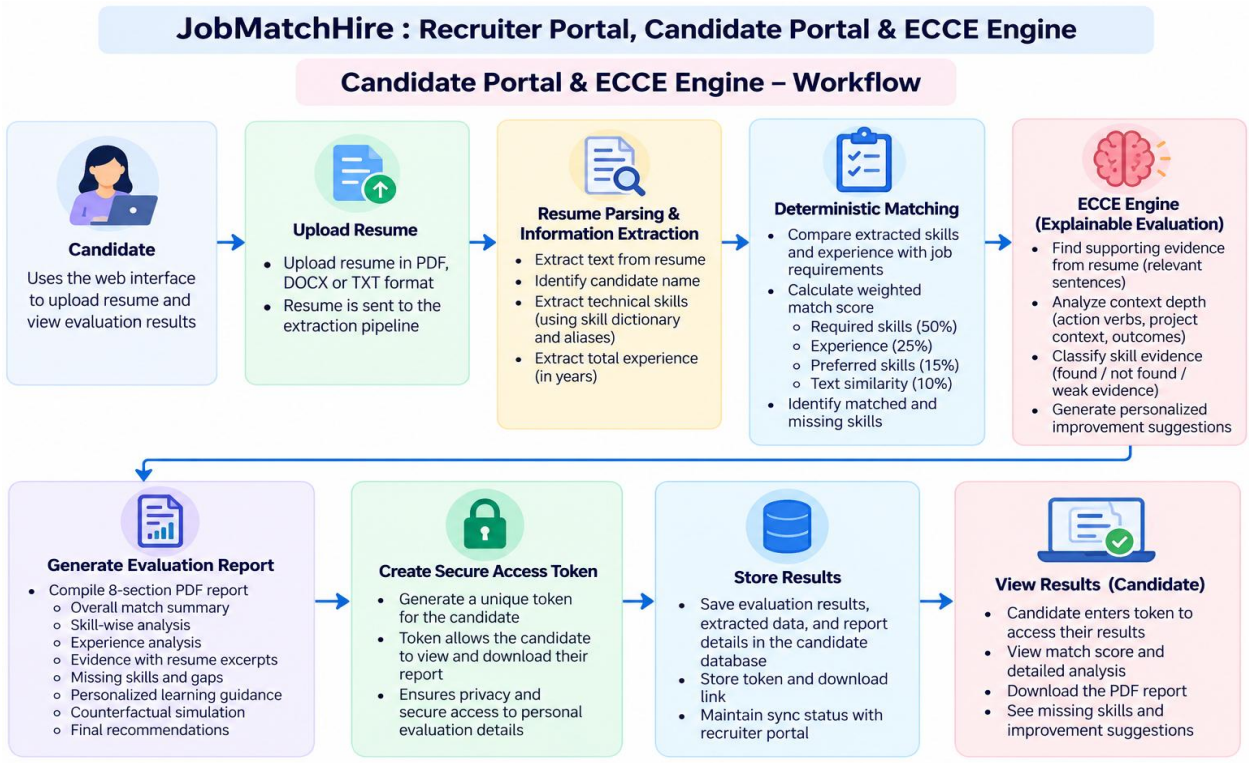
User: Candidate.
Format: Downloadable PDF (application/pdf).
API: GET /candidates/{candidate_id}/report?token={token}.
The report contains eight sections:
Job Title and Evaluation Date in IST.

Overall Match Score and Component Breakdown.
Competency Assessment.
Evidence Classification.
Verbatim Supporting Resume Passages.
Scoring Explanation and Identified Gaps.
Counterfactual Simulations.
Private Candidate Learning Guidance.
The report is protected through token-based authorization.

8.4 OVERALL INPUT-TO-OUTPUT WORKFLOW

- A recruiter creates a job requisition in the JobMatchHire Recruiter Portal.
- The JobMatchHire Recruiter Portal stores the job information in version_a.db.
- The job requisition is synchronized with the JobMatchHire Candidate Portal & ECCE Engine through a REST API.
- A candidate selects the relevant job in the JobMatchHire Candidate Portal.
- The candidate uploads a resume in PDF, DOCX, or TXT format.
- The Candidate Portal extracts the resume content and identifies relevant competencies.
- The ECCE Engine examines supporting passages and classifies competency evidence.
- The deterministic scoring engine calculates the overall match score.
- The counterfactual engine generates hypothetical improvement scenarios.
- The platform generates a private eight-section PDF evaluation report.
- The Candidate Portal & ECCE Engine sends recruiter-safe evaluation information to the JobMatchHire Recruiter Portal.
- The Recruiter Portal updates the ranked candidate leaderboard.
- This workflow allows recruiters to evaluate applicants through the Recruiter Portal while candidates independently access their detailed competency assessments and developmental feedback through the Candidate Portal.


JobMatchHire : Recruiter Portal, Candidate Portal & ECCE Engine
Recruiter Portal – Workflow
Recruiter
Uses the web interface to manage jobs and view candidate results
Create Job Posting
Enter job title
Add job description
Specify required skills
Specify preferred skills
Add experience and education requirements
Submit Job
Job is saved in the recruiter database
Job requirement is sent to Candidate Portal for evaluation
Track Synchronization
Check sync status (synced / pending / failed)
Retry synchronization if needed
Recruiter Database
Stores job postings
Stores candidate evaluation results
View Candidate Results
Open a specific job
View ranked list of candidates based on match score and relevant criteria
Ranked Candidate List
Candidates sorted by match score
Shows key information (name, score, matched skills, experience status, short summary)
View Candidate Details
Click on a candidate to see detailed evaluation summary
View evidence-based insights (relevant skills, experience, highlights)


JobMatchHire : Recruiter Portal, Candidate Portal & ECCE Engine
Candidate Portal & ECCE Engine – Workflow
Candidate
Uses the web interface to upload resume and view evaluation results
Upload Resume
Upload resume in PDF, DOCX or TXT format
Resume is sent to the extraction pipeline
Resume Parsing & Information Extraction
Extract text from resume
Identify candidate name
Extract technical skills (using skill dictionary and aliases)
Extract total experience (in years)
Deterministic Matching
Compare extracted skills and experience with job requirements
Calculate weighted match score
Required skills (50%)
Experience (25%)
Preferred skills (15%)
Text similarity (10%)
Identify matched and missing skills
ECCE Engine (Explainable Evaluation)
Find supporting evidence from resume (relevant sentences)
Analyze context depth (action verbs, project context, outcomes)
Classify skill evidence (found / not found / weak evidence)
Generate personalized improvement suggestions
Generate Evaluation Report
Compile 8-section PDF report
Overall match summary
Skill-wise analysis
Experience analysis
Evidence with resume excerpts
Missing skills and gaps
Personalized learning guidance
Counterfactual simulation
Final recommendations
Create Secure Access Token
Generate a unique token for the candidate
Token allows the candidate to view and download their report
Ensures privacy and secure access to personal evaluation details
Store Results
Save evaluation results, extracted data, and report details in the candidate database
Store token and download link
Maintain sync status with recruiter portal
View Results (Candidate)
Candidate enters token to access their results
View match score and detailed analysis
Download the PDF report
See missing skills and improvement suggestions

9. REFERENCES

[1] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," Harvard Journal of Law & Technology, vol. 31, no. 2, pp. 841–887, 2018. [Online]. Available: https://jolt.law.harvard.edu/assets/articlePDFs/v31/Counterfactual-Explanations-without-Opening-the-Black-Box-Sandra-Wachter-et-al.pdf

[2] C. Qin, H. Zhu, T. Xu, C. Zhu, L. Jiang, E. Chen, and H. Xiong, "Enhancing person-job fit for talent recruitment: An ability-aware neural network approach," in Proc. 41st Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2018, pp. 25–34. [Online]. Available: https://doi.org/10.1145/3209978.3210025

[3] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in Proc. 2020 Conf. Fairness, Accountability, and Transparency, 2020, pp. 469–481. [Online]. Available: https://doi.org/10.1145/3351095.3372828

[4] C. Zhu et al., "Person-job fit: Adapting the right talent for the right job with joint representation learning," ACM Transactions on Management Information Systems, vol. 9, no. 3, Art. no. 12, 2018. [Online]. Available: https://doi.org/10.1145/3234465

[5] A. Gugnani and H. Misra, "Implicit skills extraction using document embedding and its use in job recommendation," in Proc. AAAI Conf. Artificial Intelligence, vol. 34, no. 8, pp. 13286–13293, 2020. [Online]. Available: https://doi.org/10.1609/aaai.v34i08.7038

[6] J. Jiang, S. Ye, W. Wang, J. Xu, and X. Luo, "Learning effective representations for person-job fit by feature fusion," in Proc. 29th ACM Int. Conf. Information and Knowledge Management, 2020, pp. 2549–2556. [Online]. Available: https://doi.org/10.1145/3340531.3412717

[7] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135–1144. [Online]. Available: https://doi.org/10.1145/2939672.2939778

[8] P. Cappelli, P. Tambe, and V. Yakubovich, "Artificial intelligence in human resources management: Challenges and a path forward," California Management Review, vol. 61, no. 4, pp. 15–42, 2019. [Online]. Available: https://doi.org/10.1177/0008125619867910

[9] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988. [Online]. Available: https://doi.org/10.1016/0306-4573(88)90021-0

[10] M. Bogen and A. Rieke, Help Wanted: An Exploration of Hiring Algorithms, Equity, and Bias. Upturn, 2018. [Online]. Available: https://www.upturn.org/static/reports/2018/hiring-algorithms/files/Upturn%20--%20Help%20Wanted%20-%20An%20Exploration%20of%20Hiring%20Algorithms%2C%20Equity%20and%20Bias.pdf